

Tecnológico de Monterrey

Evidencia 2 - Actividad Integral de Estructuras de Datos Lineales

Programación de estructuras de datos y algoritmos fundamentales (Gpo 607)

Estudiante: César Adahir Cárdenas Navarro

Matrícula: A00844803

Fecha: 18/10/2026

Reflexión sobre la Evidencia 2: Estructuras de datos lineales.

Aprendizajes

En esta práctica trabajé en la implementación y pruebas de distintas estructuras de datos lineales y algoritmos para ordenar información. Más allá de revisar la teoría, analizar archivos reales de bitácora (log607-1.txt y log607-2.txt) me ayudó a entender lo importante que es saber elegir la herramienta adecuada. También me di cuenta de que un programa debe estar preparado para casos donde las cosas no salen como se espera, como cuando faltan archivos, los registros no existen o los datos vienen con fechas incorrectas.

Rol y Arquitectura de las Estructuras de Datos

Cada estructura tuvo una función muy clara en la solución:

- **Lista enlazada (LinkedList):** La usé para ir guardando todos los registros de los archivos sobre la marcha. Como su tamaño crece de forma dinámica, evitó reservar un espacio de memoria fijo desde el inicio.
- **Pila (Stack):** Me sirvió para dar la vuelta a los resultados al buscar por fecha, de modo que el último dato en entrar fuera el primero en mostrarse (LIFO).
- **Lista doblemente enlazada (DoublyLinkedList):** Guarda referencias hacia adelante y hacia atrás, lo que permite recorrer la información en ambos sentidos fácilmente.

Usar una pila para invertir los datos resulta conveniente porque ayuda a mantener esos resultados en un grupo separado y fácil de manejar. Sin embargo, procesar los elementos uno a uno para apilarlos y desapilarlos genera trabajo extra y consume más memoria. Por otro lado, la lista doblemente enlazada ya puede recorrerse hacia atrás gracias a sus enlaces hacia el elemento anterior, lo que evita copiar la información o hacer pasos adicionales.

Limitaciones

Un detalle importante que confirmé es por qué la búsqueda binaria pierde su ventaja en las listas enlazadas. En un arreglo tradicional, los datos están juntos en memoria y podemos saltar directamente al centro en un solo paso. En cambio, en una lista enlazada hay que ir avanzando nodo por nodo desde el principio para encontrar la mitad. Este recorrido previo requiere revisar los elementos de forma secuencial, por lo que la búsqueda pierde la rapidez que la caracteriza y termina siendo tan lenta como una búsqueda común.

Análisis entre Merge Sort vs. Quick Sort

Al comparar el desempeño de los dos algoritmos de ordenamiento con los datos de las bitácoras, se notó una diferencia muy clara:

- **Merge Sort:** Tuvo un comportamiento estable y rápido en ambas bitácoras (4.11 ms en log607-1.txt y 1.66 ms en log607-2.txt). Hizo un número controlado de comparaciones y mantuvo el orden original entre los registros que compartían la misma fecha.
- **Quick Sort:** Funcionó bien en el primer archivo (14.98 ms), pero en log607-2.txt sufrió un retraso considerable (1044.7 ms) y disparó las comparaciones a más de 6.5 millones. Esto sucedió porque la elección del último elemento como pivote no fue

favorable con la disposición de esos datos en particular. Además, este algoritmo no garantiza conservar el orden inicial de elementos con valores iguales.

Declaración sobre uso de IA

Durante el proceso utilicé herramientas de inteligencia artificial (ChatGPT) como apoyo conceptual para ordenar las ideas y sugerir escenarios de prueba. La revisión y validación dependieron en todo momento del trabajo manual: ejecuté el programa en la consola, verifiqué los tiempos reales de ejecución, analicé el comportamiento de Quick Sort y aseguré que el reporte reflejara fielmente los resultados obtenidos.